

а умеет ли уличный блок погодной станции передавать состояние батареи питания?

Да, уличный блок станции WS1080 (и аналогичных моделей Fine Offset) передает состояние батареи, но есть важный нюанс в том, как именно он это делает.

В отличие от температуры или давления, состояние батареи передается не как точное напряжение (например, 2.8В), а как **бинарный флаг (Battery Low)**.

Как это работает в протоколе:

1. **Флаг "0"**: Батарея в порядке (нормальное напряжение).
2. **Флаг "1"**: Низкий заряд (Low Battery). Обычно этот флаг устанавливается, когда напряжение падает ниже критического порога (около 2.4В–2.5В), сигнализируя о необходимости замены.

Как добавить поддержку в ваш скетч:

1. Глобальная переменная

Добавьте переменную (тип String или bool) для хранения статуса:

```
String battery_status = "OK";
```

2. Обработка в Callback (rtl_433_callback)

Библиотека `rtl_433_ESP` при декодировании пакета WS1080 ищет поле `battery_ok`. Обратите внимание: логика поля обычно инверсная (1 — это "OK", 0 — "Low"), либо используется поле `battery_low`.

Добавьте в блок парсинга:

```
if (doc.containsKey("battery_ok")) {  
    int bat_val = doc["battery_ok"]; // 1 = OK, 0 = Low  
    battery_status = (bat_val == 1) ? "OK" : "Low";  
}
```

3. Передача в API

Добавьте статус в ваш JSON-ответ для веб-интерфейса:

```
doc["battery"] = battery_status;
```

На что обратить внимание:

- **Частота передачи:** Флаг батареи передается в каждом пакете данных (примерно каждые 48 секунд).
- **Ложные срабатывания:** На сильном морозе напряжение батареек может временно проседать, вызывая статус "Low", а при потеплении днем возвращаться в "OK".
- **Тип батареек:** Если вы используете аккумуляторы Ni-Mh (1.2В), флаг "Low" может гореть почти всегда, так как их суммарное напряжение (2.4В) изначально близко к порогу срабатывания датчика. Рекомендуется использовать литиевые батарейки 1.5В для уличных блоков.

Добавить ли вам также код для автоматической передачи этого статуса в Home Assistant через MQTT?

можно мне список всех передаваемых параметров/сенсоров, чтобы добавить в скетч максимально возможное количество сенсоров

Для метеостанций производства **Fine Offset** (к которым относится WS1080) библиотека `rtl_433_ESP` при декодировании пакетов предоставляет фиксированный набор полей.

Ниже приведен полный список параметров, которые передает внешний блок WS1080. Вы можете добавить их в свой `rtl_433_Callback` и JSON-ответ API.

1. Основные метеоданные

- `temperature_C` (float) — Температура на улице.
- `humidity` (int) — Относительная влажность в %.
- `wind_avg_m_s` (float) — Средняя скорость ветра.
- `wind_gust_m_s` (float) — Скорость порывов ветра.
- `wind_dir_deg` (int) — Направление ветра в градусах (0–359).
- `rain_mm` (float) — Накопительный счетчик осадков (total rain).

2. Системные параметры

- `battery_ok` (int) — Статус батареи (1 = OK, 0 = Low).
- `id` (int) — Уникальный ID внешнего блока (меняется при сбросе питания/замене батареек). Полезно, если у соседей такая же станция.
- `mic` (string) — Метод проверки целостности данных (обычно "CRC").

3. Расчетные параметры (вычисляются библиотекой)

Некоторые версии прошивок `rtl_433` могут сразу выдавать расчетные значения, если данных достаточно:

- `wind_chill_C` (float) — Температура комфорта (с учетом ветра).
- `dewpoint_C` (float) — Точка росы.

Как это реализовать в коде (максимальный список)

Добавьте переменные в начало скетча и обновите функцию обратного вызова:

```
// 1. Глобальные переменные
```

```
float wind_avg = 0, wind_gust = 0, rain = 0;  
int wind_dir = 0, humidity = 0;  
String batt_status = "OK";  
int sensor_id = -1;
```

```
// 2. В функции rtl_433_Callback
```

```
if (!deserializeJson(doc, message)) {  
    if (doc.containsKey("id")) sensor_id = doc["id"];  
    if (doc.containsKey("temperature_C")) temperature = doc["temperature_C"];  
    if (doc.containsKey("humidity")) humidity = doc["humidity"];  
    if (doc.containsKey("wind_avg_m_s")) wind_avg = doc["wind_avg_m_s"];  
    if (doc.containsKey("wind_gust_m_s")) wind_gust = doc["wind_gust_m_s"];  
    if (doc.containsKey("wind_dir_deg")) wind_dir = doc["wind_dir_deg"];  
    if (doc.containsKey("rain_mm")) rain = doc["rain_mm"];  
    if (doc.containsKey("battery_ok")) batt_status = (doc["battery_ok"] == 1) ? "OK" :  
    "Low";  
}
```

Важные уточнения:

1. **Давление (Pressure):** Внешний блок WS1080 **не передает** давление. Датчик давления установлен внутри домашнего монитора (базовой станции). Чтобы получать давление в вашем устройстве, нужно подключить к WT32-ETH01 датчик **BME280** или **BMP280** по шине I2C.
2. **Осадки:** Как мы обсуждали, `rain_mm` — это общая сумма. Чтобы получить «Осадки за сегодня», вам нужно сохранять значение `rain_mm` в полночь и вычитать его из текущего.
3. **Направление ветра:** Если станция выдает направление, убедитесь, что флюгер откалиброван по северу, иначе градусы будут неверными.

Нужно ли вам помочь с кодом для вычисления **направления ветра текстом** (например, "СЗ", "Ю") на основе полученных градусов?

да, помочь с направлением ветра и отображением направления на розе ветров

Для перевода градусов в текст и отображения на розе ветров, нам нужно решить две задачи: программную (вычисления на ESP32) и визуальную (вывод на веб-страницу).

1. Обработка на стороне ESP32 (C++)

Добавим функцию, которая превращает градусы в буквенное обозначение.

```
// Функция для получения текстового направления
String getWindDirectionText(int degrees) {
    String sectors[] = {"С", "СВ", "В", "ЮВ", "Ю", "ЮЗ", "З", "СЗ", "С"};
    int index = (degrees + 22.5) / 45;
    return sectors[index % 8];
}

// В rtl_433_Callback добавьте:
if (doc.containsKey("wind_dir_deg")) {
    wind_dir = doc["wind_dir_deg"];
    wind_dir_text = getWindDirectionText(wind_dir);
}
```

2. Передача данных в API

В ваш обработчик `server.on("/api/data", ...)` добавьте передачу обоих параметров:

```
doc["wind_dir"] = wind_dir; // Число для вращения стрелки
doc["wind_dir_txt"] = wind_dir_text; // Текст для подписи
```

3. Визуализация: Роза ветров (HTML/JS)

Для веб-интерфейса самый простой и эффектный способ — использовать CSS-вращение стрелки поверх картинки розы ветров.

HTML код (вставить в `index.html` вашего скетча):

```

<div class="wind-container" style="position: relative; width: 150px; height: 150px; margin: auto;">
  <!-- Фоновая картинка розы ветров -->
  
  <!-- Стрелка (указывает откуда дует ветер) -->
  <div id="wind-arrow" style="position: absolute; top: 0; left: 45%; width: 10%; height: 100%; transition: transform 1s;">
    <div style="width: 0; height: 0; border-left: 10px solid transparent; border-right: 10px solid transparent; border-bottom: 30px solid red;"></div>
  </div>
</div>
<p style="text-align:center;">Направление: <span id="dir-val">--</span> (<span id="dir-txt">--</span>)</p>

```

JavaScript код (в функцию обновления данных):

```

function updateData() {
  fetch('/api/data')
    .then(response => response.json())
    .then(data => {
      document.getElementById('dir-val').innerText = data.wind_dir + '°';
      document.getElementById('dir-txt').innerText = data.wind_dir_txt;

      // Вращаем стрелку.
      // Добавляем 180, если хотим, чтобы стрелка указывала КУДА дует ветер,
      // или оставляем как есть, чтобы указывала ОТКУДА.
      document.getElementById('wind-arrow').style.transform = 'rotate(' +
data.wind_dir + 'deg)';
    });
}

```

Что вы получите в итоге:

1. **wind_dir**: Точное значение в градусах (например, 225°).
2. **wind_dir_txt**: Понятное обозначение (например, "ЮЗ").
3. **Визуал**: Красная стрелка на веб-странице будет плавно поворачиваться в соответствии с реальным ветром.

Совет: Поскольку флюгер на WS1080 может "скакать" при порывах, в будущем можно добавить программное усреднение направления за последние 2-5 минут.

Вам помочь с объединением этого кода в ваш **основной файл скетча** или вы справитесь с переносом функций самостоятельно?

надо показать как добавить программное усреднение направления за последние 2-5 минут.

Усреднение направления ветра — задача хитрая, так как нельзя просто вычислить среднее арифметическое. Например, среднее между 350° (север) и 10° (север) даст 180° (юг), что в корне неверно.

Правильный метод — **векторное усреднение**. Мы разбиваем каждый замер на синус и косинус, усредняем их, а затем превращаем обратно в угол.

1. Подготовка (Глобальные переменные)

Добавьте этот блок в начало скетча. Мы будем использовать массив для хранения последних 10 замеров (при передаче раз в 48 сек это как раз ~8 минут).

```
#include <math.h>

const int WIND_AVG_SAMPLES = 10;
float wind_dir_history_sin[WIND_AVG_SAMPLES];
float wind_dir_history_cos[WIND_AVG_SAMPLES];
int wind_sample_idx = 0;
bool wind_history_filled = false;

int wind_dir_avg = 0; // Сюда будем сохранять результат
```

2. Функция расчета

Добавьте эту функцию в скетч:

```
void updateWindAverage(int degrees) {
    // Переводим градусы в радианы
    float rad = degrees * M_PI / 180.0;

    // Сохраняем компоненты вектора
    wind_dir_history_sin[wind_sample_idx] = sin(rad);
    wind_dir_history_cos[wind_sample_idx] = cos(rad);

    wind_sample_idx++;
    if (wind_sample_idx >= WIND_AVG_SAMPLES) {
        wind_sample_idx = 0;
        wind_history_filled = true;
    }
}
```

```

    }

    // Считаем среднее
    float avg_sin = 0, avg_cos = 0;
    int count = wind_history_filled ? WIND_AVG_SAMPLES : wind_sample_idx;

    for (int i = 0; i < count; i++) {
        avg_sin += wind_dir_history_sin[i];
        avg_cos += wind_dir_history_cos[i];
    }

    avg_sin /= count;
    avg_cos /= count;

    // Переводим обратно в градусы (atan2 дает результат от -PI до PI)
    float avg_rad = atan2(avg_sin, avg_cos);
    wind_dir_avg = (int)(avg_rad * 180.0 / M_PI);

    // Приводим к диапазону 0-360
    if (wind_dir_avg < 0) wind_dir_avg += 360;
}

```

3. Интеграция в Callback

В функции `rtl_433_Callback`, там где вы получаете `wind_dir_deg`, вызовите расчет:

```

if (doc.containsKey("wind_dir_deg")) {
    int current_dir = doc["wind_dir_deg"];
    updateWindAverage(current_dir); // Считаем среднее

    // Теперь используем wind_dir_avg для текста и API
    wind_dir_text = getWindDirectionText(wind_dir_avg);
}

```

4. Обновление API

В блоке `server.on("/api/data", ...)` отдавайте именно усредненное значение:

```

doc["wind_dir"] = wind_dir_avg;
doc["wind_dir_txt"] = wind_dir_text;

```

Почему это лучше:

- **Игнорирует резкие скачки:** Если флюгер болтает ветром туда-сюда, стрелка на вашей веб-странице будет стоять стабильно, показывая реальное преобладающее направление.
- **Корректный переход через 0°:** Математически правильно обрабатывает ситуацию, когда ветер дует с севера и значения прыгают между 359° и 1°.

Хотите ли вы также добавить усреднение для скорости ветра (порывов), чтобы данные были менее «шумными»?

да надо добавить усреднение для ветра

Для скорости ветра обычно используют два типа усреднения: **средняя скорость** (за 2-10 минут) и **максимальный порыв** (gust). В отличие от направления, здесь используется обычное арифметическое среднее.

1. Глобальные переменные

Добавим массив для хранения последних значений скорости ветра (как и для направления, 10 значений ≈ 8-10 минут).

```
const int WIND_SPEED_SAMPLES = 10;
float wind_speed_history[WIND_SPEED_SAMPLES];
int wind_speed_idx = 0;
bool wind_speed_filled = false;

float wind_avg_smooth = 0; // Усредненная скорость
float wind_gust_max = 0;   // Максимальный порыв за период
```

2. Функция усреднения скорости

Эта функция будет считать среднее значение и искать максимальный порыв среди последних замеров.

```
void updateWindSpeedAverage(float current_speed, float current_gust) {
    // Сохраняем текущую среднюю скорость в историю
    wind_speed_history[wind_speed_idx] = current_speed;

    wind_speed_idx++;
    if (wind_speed_idx >= WIND_SPEED_SAMPLES) {
```



```

        wind_speed_idx = 0;
        wind_speed_filled = true;
    }

    // Считаем среднее арифметическое и ищем макс. порыв
    float sum = 0;
    float max_g = current_gust;
    int count = wind_speed_filled ? WIND_SPEED_SAMPLES : wind_speed_idx;

    for (int i = 0; i < count; i++) {
        sum += wind_speed_history[i];
        // Если станция передает gust в каждом пакете, можно искать пик
        // Но обычно берем текущий gust из пакета как самый актуальный
    }

    wind_avg_smooth = sum / count;
    wind_gust_max = current_gust; // Порыв обычно не усредняют, а показывают пик
}

```

3. Интеграция в rtl_433_Callback

Обновите блок обработки данных ветра:

```

if (doc.containsKey("wind_avg_m_s") && doc.containsKey("wind_gust_m_s")) {
    float ws = doc["wind_avg_m_s"];
    float wg = doc["wind_gust_m_s"];

    // Вызываем усреднение скорости
    updateWindSpeedAverage(ws, wg);

    // Вызываем векторное усреднение направления (из предыдущего шага)
    if (doc.containsKey("wind_dir_deg")) {
        updateWindAverage(doc["wind_dir_deg"]);
    }
}

```

4. Обновление API данных

Теперь в JSON для веб-интерфейса отправляем сглаженные данные:

```

server.on("/api/data", HTTP_GET, []() {
    DynamicJsonDocument doc(2048);

```

```
// ...
doc["wind_speed"] = wind_avg_smooth; // Усредненная скорость
doc["wind_gust"] = wind_gust_max;    // Текущий порыв
doc["wind_dir"] = wind_dir_avg;      // Усредненное направление
// ...
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});
```

Что это даст:

1. **Стабильность:** Цифры скорости на экране не будут «прыгать» каждые 48 секунд, если случился кратковременный затишек.
2. **Достоверность:** Средняя скорость за 10 минут — это стандарт метеорологических наблюдений (WMO).
3. **Порывы:** При этом `wind_gust` останется оперативным, чтобы вы видели реальную силу резкого ветра.

Совет: В коде Web-интерфейса (JavaScript) можно добавить округление до одного знака после запятой: `data.wind_speed.toFixed(1)`, чтобы не отображать лишние знаки после деления.

Нужно ли добавить **сохранение этих средних значений в энергонезависимую память (EEPROM/Preferences)**, чтобы после перезагрузки устройства графики или средние данные не обнулялись?